## WHAT A DATABASE IS

**Database / DBMS?** DB = possibly large collection of shared data that may **never get lost or corrupted**. DBMS = the **software** that manages it and keeps it safe (PostgreSQL).

**Why a DBMS, not files?** Files can't cope with many users at once, crashes or wrong data. A DBMS handles many users at the same time (concurrency), keeps data correct (integrity rules, ACID), recovers after crashes, controls who may see what, and offers a declarative language.

**"Information is an asset": requirements? Availability** (always reachable) · **reliability** (correct, even after crashes) · **performance** (many transactions/sec) · **durability/scalability** (structure can change without breaking apps) · **security**. Downtime or bad data costs money.

**Roles?** End user (uses apps) · application programmer (writes the SQL apps) · information analyst (requirements, ER model) · database designer (tables, rules) · DBA (runs, tunes, secures, backs up the DBMS).

## ARCHITECTURE

**ANSI/SPARC and data independence?** 3 levels: **external** schemas (a view per app), one **conceptual** schema (all tables), one **physical** schema (disk, indexes). **Physical independence:** change storage, tables unchanged. **Logical:** change tables, app views keep working. *Why:* apps needn't be rewritten.

**Index? Pro and con?** Like the index at the back of a book: an extra sorted search structure (B-tree) pointing to rows. **+** finds rows fast without reading the whole table (table scan). **−** extra storage and every insert/update must also update the index. Physical level: queries don't change.

**Concurrency, transparency, replication, distribution?** What the DBMS hides from users: many users at the same time (concurrency) · you don't see how or where data is stored (transparency) · copies of the data kept for safety and speed (replication) · data spread over several computers (distribution).

**Query processing?** SQL → relational algebra → optimized (reorder, choose indexes) → **query plan** → executed. This is why SQL can be declarative.

**Data dictionary / catalog?** Where the DBMS stores metadata (schemas, rules, users), itself as tables you can query.

## RELATIONAL MODEL

**What makes a DB "relational"?** Data is stored in **tables (relations)** of rows and typed columns, and the tables are **linked by keys**: a foreign key holds the primary key value of a row in another table. "Foreign keys are what makes relational DBs relational." Each fact is stored once; joins rebuild the full picture.

**Record, attribute, domain?** Record/tuple = a row. Attribute = a column. Domain = the allowed values (type) of an attribute.

**Schema vs instance?** Schema = the structure: table names, columns and their types (the empty form, rarely changes). Instance = the actual data in it right now (the filled-in forms, changes all day).

**NULL?** Unknown / not applicable / missing. Not 0, not ''. "= NULL" is never true; test with IS NULL.

**Atomic values: why?** One value per cell, no lists. With lists you need a complex language with programming constructs and can't filter/index the parts. Fix: one row per value.

## KEYS AND INTEGRITY

**Role of a primary key? Uniquely identifies each row**: its value is unique and never NULL, and every table has exactly one. *Why:* it stops duplicate rows and gives foreign keys something to point at. E.g. boxer_id, s-number.

**Composite key?** PK of 2+ attributes: each part may repeat, the **combination** may not (one grade per student per course).

**Candidate / surrogate / natural key?** Candidate = any attribute set that could be PK. Surrogate = generated number (SERIAL id) when natural keys are long, composite or change. Natural = real value (email).

**Why can a PK never be NULL?** Unknown can't identify a row or be checked for uniqueness; FKs couldn't point to it.

**Foreign key and referential integrity?** FK = attribute referring to the key of another table. Referential integrity = every FK value **exists** as a key (or is NULL); the DBMS rejects breaking changes. E.g. trainer_id 99 with no trainer 99 fails.

**Kinds of integrity rules? Key** (no two rows same key) · **foreign key** (refers to an existing key) · **semantic** (business rule / CHECK, e.g. grade 1–10).

**NOT NULL vs UNIQUE on an FK?** Far side exactly 1 → NOT NULL; 0..1 → nullable. Own side 1 or 0..1 (1-1) → UNIQUE; * → not unique.

## REDUNDANCY

**Redundancy: what and why bad?** The same fact stored more than once (a department's name on every employee row). Bad because it wastes space and causes **inconsistency**: you update one copy, forget another, and the DB contradicts itself (update / insert / delete anomalies). Fix: split into tables linked by keys so each fact is stored once.

**Why decompose one big table?** Each fact stored once, no contradictions; cost = queries must join, but redundancy is worse (normalization).

**Why FK on the * side of 1—*?** Each "many" row belongs to exactly one "one" row, so it stores one key. The other way repeats department data on every employee row (high redundancy, high update cost, fragile). FK side: every fact stored once.

## TRANSACTIONS AND ACID

**Why transactions?** A transaction = several operations that logically belong together, treated as **one unit**. Needed because many users change data at the same time (**concurrency**) and crashes happen; without it data gets corrupted. Bank: two interleaved transfers turn 3000 into 3700 (**lost update**). The DBMS gives every transaction the ACID guarantees.

**A · Atomicity** All or nothing; on failure everything is rolled back. Transfer: debit done, crash → debit undone.

**C · Consistency** Before and after, all integrity rules (PK, FK, CHECK) hold; a violating transaction is reversed.

**I · Isolation** Concurrent transactions don't see each other's unfinished work; as if each had the DB alone.

**D · Durability** After COMMIT the result survives anything: crash, network loss, disk failure.

**COMMIT, ROLLBACK, recovery?** COMMIT = make permanent. ROLLBACK = undo all. Recovery = after a crash use the log to undo unfinished and finish committed transactions; backups for disk loss.

## SQL CONCEPTS

**Why is SQL declarative?** You only say **what** result you want, not **how** to get it (no loops, no "read this file first"). The DBMS optimizer chooses the plan (index or full scan, join order). *Why good:* the fastest plan is chosen for you and queries keep working when storage changes.

**Composable?** Every query returns a table, so queries nest inside queries (subqueries).

**DDL, DML, SDL? DDL = Data Definition Language**: defines structure (create/alter/drop). **DML = Data Manipulation Language**: works on data (select/insert/update/delete). **SDL = Storage Definition Language**: storage (indexes): changes speed, never results.

**Projection vs selection?** Projection = choose **columns**. Selection = choose **rows** by a condition.

**Forget the join condition? Cartesian product**: every row × every row (100 × 10 = 1000); meaningless and very slow.

**Why is a join needed at all?** Facts are split over tables; a join recombines them through FK = PK.

**Aggregate, GROUP BY, ORDER BY, LIMIT? Aggregate function** = one value computed over many rows (COUNT, SUM, AVG, MIN, MAX). **GROUP BY** = makes one summary row per group of equal values ("average rating per year"). **ORDER BY** = sorts the result (ascending unless DESC). **LIMIT** = keep only the first n rows.

**JOIN vs LEFT JOIN?** JOIN keeps matches only; LEFT JOIN keeps all left rows, NULL where no match ("also with zero").

**Self-join?** A table joined with itself (two aliases) to compare rows: same year as Alien, teacher and manager, pairs.

**WHERE vs HAVING?** WHERE filters rows **before** grouping (totals don't exist yet); HAVING filters groups **after**. Never an aggregate in WHERE.

**COUNT(*) vs COUNT(col)?** COUNT(*) counts rows; COUNT(col) skips NULLs; COUNT(DISTINCT col) counts different values.

**Nested query?** Query inside a query; possible because SQL is composable.

**View: what and why?** A stored query used like a table. Why: limited access rights, simpler queries, efficiency.

**CHECK vs FK?** CHECK = condition on the row's own values; FK = value must exist in another table.

## SQL CODE

```
CREATE TABLE course (
  course_id  INT PRIMARY KEY,
  title      VARCHAR(80) NOT NULL,
  email      VARCHAR(100) UNIQUE,
  credits    DECIMAL(3,1),
  start_date DATE,
  teacher_id INT NOT NULL,          -- exactly 1
  FOREIGN KEY (teacher_id) REFERENCES teacher(teacher_id)
);
CREATE TABLE enrolled (           -- * to * middle table
  student_id INT, course_id INT, grade DECIMAL(3,1),
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES student(student_id),
  FOREIGN KEY (course_id)  REFERENCES course(course_id)
);
ALTER TABLE student ADD COLUMN phone VARCHAR(20);
DROP TABLE enrolled;

INSERT INTO teacher (teacher_id, name) VALUES (1, 'Ann');
INSERT INTO teacher VALUES (2,'Bob',NULL), (3,'Kim','k@ut.nl');
UPDATE enrolled SET grade = grade + 0.5 WHERE course_id = 10;
UPDATE course SET credits = 6
WHERE teacher_id = (SELECT teacher_id FROM teacher
                    WHERE name = 'Ann');
DELETE FROM enrolled WHERE grade IS NULL;

SELECT name FROM teacher UNION     SELECT name FROM student;
SELECT name FROM teacher UNION ALL SELECT name FROM student;
SELECT name FROM teacher INTERSECT SELECT name FROM student;
SELECT name FROM teacher EXCEPT    SELECT name FROM student;
```

**Create the referenced table first** (teacher before course). No comma after the last line. Text in 'single quotes', apostrophe twice ('Chef''s'). NULL is tested with IS NULL. No WHERE = every row. **UNION** = in either, no duplicates · **UNION ALL** keeps duplicates · **INTERSECT** = in both · **EXCEPT** = in first, not in second. Both sides: same number of columns, matching types.

## SQL EXERCISES: QUESTION WORD → WHAT TO WRITE

| Question says | Use |
|---|---|
| "pairs of …", "two … with the same …", combinations | self-join, **a.id < b.id** (each pair once) |
| "other … than X", "except X itself", "a different …" | **<>** (not itself): title <> 'Barbie' |
| "more than one", "at least 2" | GROUP BY key HAVING COUNT(*) > 1 |
| "different …" (doctors, customers) | COUNT(**DISTINCT** col) |
| "for each / per …" | GROUP BY (every non-aggregate column) |
| "total / average / how many" | SUM / AVG / COUNT |
| "also with zero / none" | LEFT JOIN + COUNT(col), not COUNT(*) |
| "never / no / not any" | NOT EXISTS, NOT IN or EXCEPT |
| "both A and B" / "A or B" | INTERSECT / UNION |
| "same … as X" | = (SELECT … WHERE name = 'X') |
| "highest / oldest / most" | = (SELECT MAX/MIN …); LIMIT 1 hides ties |
| name given, table stores the id | JOIN, or IN (SELECT id … WHERE name = …) |
| condition on a count/sum vs on a row | HAVING vs WHERE |
| change a row / add a row | UPDATE … SET … WHERE / INSERT INTO |

**Self-joins and comparisons:**

| You want | Use | Result for A, B |
|---|---|---|
| Pairs, each pair once | a.id < b.id | A–B |
| Pairs in both directions | a.id <> b.id | A–B and B–A |
| Nothing extra | only = | A–A, A–B, B–A, B–B ✗ |

**< = "each pair once"** (pairs, combinations). **<> = "not itself"** (other, a different …). One clue decides it: "pairs of …" → <; "other … than X" → <>. Never type values from the data (ids, emails) unless the question gives them.

## DESIGN

**Why a class model before tables?** Design in 2 steps: class model, then tables. Users can check the model; it stays future-proof and independent of implementation. Don't design for efficiency; aim for fidelity.

**ER modelling?** Entity-Relationship modelling: drawing the information as entities (boxes) and relationships (lines) before building tables. You must be able to read an ER model and turn it into a table structure.

**ER words?** Entity/class = a kind of thing (a box: Student) · attribute = a property of it (name) · <<KEY>> = the identifying attribute · <<SET>> = an attribute with several values · association = a line = a relationship (takes) · role = name on a line when a class appears twice (from/to) · association class = dashed box with attributes of the link itself (grade).

**Multiplicity?** The numbers at the ends of a line saying **how many**: 1 = exactly one, 0..1 = zero or one (optional), * = zero or more, 1..* = at least one. Read it from the far side: "1 Teacher --- * Course" = each course has exactly 1 teacher, a teacher has many courses. It decides where the FK goes and NOT NULL / UNIQUE.

**Standard approach?** Each class → table · each table → PK (add an ID if needed) · each relation → FK or a separate table · IS-A → choose an option.

**Many-to-many?** Both sides have "many" (a student takes many courses, a course has many students), so neither table can store the other's key (a list in one cell is not allowed). Solution: a **middle table** with both FKs, PK(both FKs) and the attributes of the link.

**1-1?** FK on one side only, with UNIQUE (FKs both ways duplicate and can contradict) or merge the tables.

> **IS-A → FK in the subclasses:** sub PK = super PK, also FK to super: Student(**member_id** PK, FK → Member). Super gets no FK. Other options: only subs · one table + type column.

**Reflexive association?** A class linked to itself (manager is a teacher) → FK to its own table.

**<<SET>> attribute?** Own table: owner key + value, PK(both).

> **Association class → middle table, composite PK** = the two classes' PKs, each also an FK; its attributes are normal columns: Loan(**member_id, copy_id**, date). Pair can repeat → date in PK.

**Surrogate key on a middle table?** Then still add UNIQUE(fk1, fk2), or the same pair can be stored twice.

| Line (FK in B) | FK column gets | Example |
|---|---|---|
| A 1 --- * B | **NOT NULL**, not unique | course.teacher_id |
| A 0..1 --- * B | nothing (NULL ok) | patient.room_nr |
| A 1 --- 0..1 B | **NOT NULL UNIQUE** | bracelet.patient_id |
| A 0..1 --- 0..1 B | **UNIQUE** | person.partner |
| A * --- * B | middle table, PK(both) | likes(user, song) |

**Composite FK?** Several columns together referencing a composite PK; half the key isn't enough.

## TYPES, OLTP/OLAP

**Choosing types?** TEXT (any length) · VARCHAR(n) (≤ n) · CHAR(n) · INTEGER · SERIAL (auto id) · NUMERIC(n,d) exact (money, grades) · DOUBLE PRECISION inexact · DATE · TIMESTAMP · BOOLEAN. A type is an integrity rule.

**OLTP vs OLAP?** OLTP: daily operations, many small writes, normalized, ACID (registration). OLAP: analysis, few writes, big reads, denormalized star/snowflake (sales warehouse).

**Star schema?** Central fact table + dimension tables (time, store, product); built for fast aggregation.

## ABBREVIATIONS

**Databases:** DB = DataBase · DBMS = DataBase Management System · DBA = DataBase Administrator · SQL = Structured Query Language · DDL / DML / SDL = Data Definition / Data Manipulation / Storage Definition Language · PK / FK = Primary / Foreign Key · ACID = Atomicity, Consistency, Isolation, Durability · OLTP / OLAP = On-Line Transaction / Analytical Processing · ER(D) = Entity-Relationship (Diagram) · UML = Unified Modeling Language · CRUD = Create, Read, Update, Delete · ANSI/SPARC = the standards committee behind the 3-level architecture.

**Machine learning:** AI / ML = Artificial Intelligence / Machine Learning · KNN = k-Nearest Neighbours · MLP = MultiLayer Perceptron · TP, FP, FN, TN = True/False Positive/Negative · MAE = Mean Absolute Error · MSE = Mean Squared Error · RMSE = Root Mean Squared Error · ReLU = Rectified Linear Unit · sgn = sign function · sklearn = scikit-learn (Python ML library) · pd / np = pandas / numpy · CSV = Comma-Separated Values.

## ML SYMBOLS, MISTAKES, PROJECT

**Common mistakes?** Averaging for a Pass/Fail question (vote!) · counting the point itself as its own neighbour · ignoring a tie at the last seat · keeping labels 0/1 in the perceptron (use −1/+1) · updating at the end of the epoch (update immediately) · sgn(0) = +1 (it is −1) · MAE without dropping minus signs · swapping precision and recall · scaling/imputing before the split · numbering a nominal category · keeping an ID column · choosing k on training data.

**Symbols?** x = input · y = true answer · $\hat{y}$ (y-hat) = prediction · w = weight · b = bias · z = weighted sum (or z-score) · η (eta) = learning rate · μ (mu) = mean · σ (small sigma) = standard deviation · Σ (capital Sigma) = "add it all up" · k = number of neighbours.

**Project and exam?** Project: teams of 4 build a Streamlit web app on a dataset (explore with business rules, SQL-like queries on pandas DataFrames, graphs, a trained ML model); pass/fail. Exam: 1h30; the diamond grade is the exam grade.

## AI AND ML

**What is AI?** Systems doing tasks needing human intelligence: learning, reasoning, language, recognizing patterns, deciding.

**Model, feature, label, example? Model** = what learning produces: the learned "program" (new data in, prediction out). **Feature** (attribute) = an input column (study hours). **Label / target** = the answer column to predict (Pass/Fail, rent). **Example / instance / data point** = one row.

**3 types of AI? Rule-based**: people write IF-THEN rules. **Probabilistic**: experts give probabilities. **Data-driven** = ML: learns from examples.

**Limits of rule-based?** Only yes/no, can't learn, rules and exceptions explode, can't find a rule nobody thought of. Transparent but fragile.

**Limits of probabilistic?** Needs accurate probabilities nobody has; bad estimates → bad results; explodes with many variables; still hand-encoding.

**ML vs programming?** Programming: input + logic → output. ML: input + output → logic (the model). Adapts when patterns change.

**Supervised / unsupervised / reinforcement? Supervised** = learns from examples that come **with the right answer** (labels), like studying with an answer sheet; then predicts answers for new cases (spam filter, pass/fail, price). **Unsupervised** = gets data **without answers** and finds groups or patterns itself; a human names them (customer types). **Reinforcement** = no dataset: an agent **tries actions** and learns from rewards and penalties (game AI, robot learning to walk). Test: answer column → supervised; none → unsupervised; only rewards → reinforcement.

**Classification vs regression?** Both are supervised. **Classification** = predict a **label / category** from a fixed list ("which box?": pass/fail, spam/not spam, which digit); the model learns a **boundary** between classes. **Regression** = predict a **number** on a scale ("how much?": score 63.7, price, temperature); the model learns a line/**curve**. Test: is the answer a label or a number? A digit 0–9 is still a label; age group = classification, age in years = regression.

**ML project steps?** Problem → data → cleaning → training → evaluation (loops back) → deployment. Most time in data.

**Hyperparameter vs parameter?** Chosen by you (k, learning rate, epochs) vs learned (weights).

**Generalization?** Doing well on **new, unseen** data, not just the training data. The whole goal of ML; the test set measures it.

## KNN

**How KNN works** KNN = k-Nearest Neighbours. To predict a new point: **1** measure its distance to **every** stored example · **2** sort from closest to farthest · **3** keep the k closest (k = how many neighbours vote, you choose it) · **4** classification: majority vote of their labels; regression: average of their values. Idea: similar cases end alike. E.g. k = 3, neighbours Pass, Pass, Fail → Pass.

**Lazy learner?** KNN doesn't learn anything in advance: no training, it just stores all examples and does the work when asked to predict. Good: nothing to train, new data is used immediately. Bad: slow and memory-heavy on big data.

**Euclidean vs Manhattan?** Straight line vs along the axes (city blocks). The choice can change the neighbours.

**k = 1?** Copies one point: fooled by noise, 100% on training data (self-praise).

**Choosing k?** Try several on held-out / validation data or cross-validation; never on training data.

**Weighted KNN?** Closer neighbours get bigger votes; a few close ones can beat many far ones.

**Noise?** Noise = data that is **wrong** and does not show the real pattern: a label typed in error (a failing student marked Pass) or a wrongly measured value. A noisy point sits **inside the wrong class**. A borderline point is different: correct, but sitting between two classes. Why it matters: with k = 1 one noisy neighbour decides; a bigger k lets correct neighbours outvote it.

**Strengths / weaknesses?** + simple, no training, class and number, any shape. − slow, stores all, needs scaling, hurt by noise and irrelevant features, k to choose.

**Curse of dimensionality?** Many features → sparse space; nearest points aren't near; distances lose meaning. Fix: fewer, better features.

**Irrelevant features?** Unrelated to the answer but still change distances (shoe size, music style). Drop them.

**Classification vs regression in KNN?** Same neighbours; vote vs mean.

## EXPLAIN: WHAT HAPPENS IF ... AND WHY

**Include all tied neighbours?** More than k vote (k = 3 → 4 vote). Fair: equally close = equally similar. But it can make a vote tie (2–2) → then a vote-tie rule is needed.

**"First listed" rule?** Keeps exactly k, but the winner depends on the data order (arbitrary). scikit-learn does this.

**Tie inside / outside the top k?** No effect. Only a last-seat tie with different labels matters → show both outcomes, state the rule.

**k = 1 with two equally near points?** Different labels → 1-NN can't decide → rule (first listed / next neighbour).

**Vote tie?** Prevent: odd k (2 classes). Fix: closest class wins, weighted vote (1/d), nearest decides, or k − 1. k > 1: one wrong neighbour is outvoted.

**k = all points?** Always the overall mean / majority class: far points vote like near ones, locality is lost.

**ID column kept?** Meaningless number that changes distances / gets memorized → drop it.

**One feature has a much bigger range?** It dominates the distance (RAM 4–32 vs screen 11–16) → scale (min-max or standardize), fitted on train only.

**Binary (0/1) features + Euclidean?** Works (same scale): √(number of mismatches). Hamming is more natural; many ties.

**Perceptron: z = 0? input 0?** sgn(0) = −1, so a +1 point with z = 0 is wrong → update. An input of 0 leaves its weight unchanged (0 × anything).

## ENCODING AND SCALING

**Why encode categories?** Distances need numbers.

**Why not number them 1, 2, 3?** Fake order and fake distances; renumbering flips the answer.

**One-hot: why it works?** One 0/1 column per category (major_CS, major_Econ, major_Psych); a row has 1 only in its own category. Every two different categories differ in exactly 2 columns, so all are **equally far apart**: no fake order, no fake "halfway".

**Nominal vs ordinal?** No order → one-hot. Real order (BSc<MSc<PhD) → codes, but the gaps are a judgement KNN believes.

**Music style problem (S3)?** Category stored as a number + its range dominates + probably irrelevant → one-hot or drop, and scale.

**Why scale for KNN?** KNN adds up the differences of all features. If one feature has big numbers (minutes 330–480, sq ft 1200–2400) and another small ones (hours, bedrooms 2–4), the big one **owns the distance** and the small one is ignored, only because of its unit. Scaling = putting every feature on the same kind of ruler so each gets a fair, equal say.

**Standardization vs min-max? Standardization (z-score)**: subtract the average, divide by the standard deviation → "how many typical spreads above/below average"; result has mean 0, spread 1; the default for KNN. **Min-max** (normalization): "where between lowest and highest" → squeezes everything into 0–1; one extreme outlier squashes all the others together.

**min-max:** $x' = (x - \min) \div (\max - \min) \rightarrow 0 \ldots 1$ · **z-score:** $z = (x - \mu) \div \sigma \cdot \mu = \Sigma x \div n$ (mean) · $\sigma = \sqrt{\Sigma (x - \mu)^2 \div n}$ (std): subtract μ from each x, square, add up, ÷ n, √.

**Why fit scaling on training only?** Same ruler for new points, and no leakage of test data.

**Does a perceptron need scaling?** Works without (weight per feature) but converges much faster with it (raw apartments: 12,611 epochs).

## EVALUATION

**Why train/test split?** Grade on unseen data, like an exam. Training score = self-praise.

**Validation set / cross-validation?** Extra split to tune k so the test stays clean / rotate held-out folds and average: more reliable.

**Accuracy misleads when?** Accuracy = correct predictions ÷ all predictions. It lies when one class is much bigger (**imbalanced data**): 100 payments with 5 fraud, a lazy model that always says "benign" gets 95% accuracy but catches **0** fraud. So compare with what "always guess the majority" scores, and use the confusion matrix, precision and recall.

**Confusion matrix?** A 2×2 table of predicted vs actual. **TP** = said positive, was positive (caught) · **FP** = said positive, was negative (false alarm) · **FN** = said negative, was positive (missed) · **TN** = said negative, was negative (correctly ignored). Positive = the class you hunt for (fraud, COVID, Pass), not "good". Read the name backwards: False Negative = said negative, wrongly.

**Precision vs recall? Precision** = of everything the model **flagged** positive, how much was really positive (hates false alarms). **Recall** = of all **real** positives, how many it caught (hates misses). Flag everything → recall 100%, precision low; flag only when sure → precision high, recall low. Choose by the cost of the mistake: illness/fraud → recall; blocking innocent customers → precision.

**MAE vs RMSE?** Both measure how far off a regression model is (residual = true − predicted). **MAE = Mean Absolute Error**: drop the minus signs, take the average: "on average we are X off", in the task's own units. **RMSE = Root Mean Squared Error**: square the errors, average, take the root; squaring makes **big mistakes count extra**, so RMSE ≥ MAE always. Absolute values/squares stop +5 and −5 cancelling to a fake 0. Use RMSE when one big miss is worse than several small ones.

**Residual?** True minus predicted.

**Imbalanced data?** Report precision/recall for the rare class, weight/resample, pick threshold by cost.

**Why training error isn't enough?** It measures memory, not generalization.

**Medical test metric?** Recall: a missed patient is worse than a false alarm.

## PERCEPTRON

**What is a perceptron?** The simplest neural network: **one artificial neuron** that sorts things into two classes (+1 / −1). It looks at the inputs (e.g. sleep and study hours), gives each a **weight** (how important), adds them up with a **bias**, and says +1 if the total is above 0, otherwise −1. Geometrically it draws **one straight line** between the two groups. Unlike KNN it really learns: after training it only needs its weights and bias.

**Parts?** Inputs · a weight per input · bias · weighted sum · step function (positive → +1, else −1) · output.

**Weights / bias geometrically?** Weights tilt the line, bias shifts it.

**How does it learn?** It goes through the training points one by one. Correct → change nothing. Wrong → push the weights and bias towards the right answer (add the inputs for a missed positive, subtract them for a false positive), immediately, before the next point. Repeat epochs until a whole epoch has no mistakes.

**Learning rate?** Step size: too big jumps around, too small is slow.

**Epoch / convergence?** Epoch = one full pass over all training points. Convergence = reaching an epoch with zero updates (all points correct); then training stops.

**sgn(0)?** −1, so all-zero weights predict −1 at first.

**Linearly separable?** One straight line splits the classes → guaranteed to converge. Not separable → loops forever: stop at max epochs or use another model.

**Perceptron vs KNN?** Learns weights (cheap prediction, a weight per feature → copes with unscaled/irrelevant features) vs stores data (any shape, needs scaling).

**vs MLP neuron?** Step (±1, one line) vs sigmoid/ReLU in layers (curved boundaries).

## OTHER MODELS AND BOOK

**Linear regression?** Draws the **best straight line through the points** and reads the answer off it, to predict a **number** (score from hours studied). Same formula as the perceptron but without the +1/−1 step. "Best" = the line with the smallest total squared error. Weak: can't follow curves, outliers pull it, nonsense far outside the data.

**Decision tree?** Flowchart of feature questions; leaves answer. Gini 0 = pure node.

**Choosing a model?** Label or number, linear or not, data size, need to explain; compare on held-out data.

**MLP / forward propagation?** Layers input → hidden → output (one per class), fully connected; pass the example forward layer by layer.

**Sigmoid / vanishing gradient?** S-curve into 0–1. Deep sigmoid nets: learning signal shrinks each layer → early layers stop learning; ReLU fixes.

**Universality theorem?** An MLP can approximate any function, but it doesn't say how to find the weights.

**pos / neg?** The 2 class names: the thing you look for vs not.

**Hamming / mixed distance?** Count differing positions; mixed: numbers squared difference, categories 0 same / 1 different.

**Distance rules?** ≥ 0, 0 to itself, symmetric, triangle inequality.

**Tomek link / consistent subset?** Mutual nearest neighbours of different class → remove (noise). Smaller set giving the same answers (drop redundant points).

**Kangaroo lesson?** One attribute (pouch) can matter most and relations between attributes matter; KNN weighs all equally → need models that build features.

## DATA CLEANING AND FEATURES

**Outlier?** A value **extremely far** from the others, usually a data-entry error (age 250, income 1,200,000). Harmful: it pulls distances, lines and averages towards itself, so KNN's boundary shifts and feature importance gets corrupted.

**Inconsistent format?** The same kind of value written in different ways: "sixty thousand" vs 60000, "71000" vs "71,000", "Yes" vs "yes". The computer sees them as different values, so fix them to one format.

**Missing / impossible value / duplicate? Missing** = no value (NULL). **Impossible** = can't exist in reality (age −5). **Duplicate** = the same row entered more than once (counts 5× in training).

**Why clean data?** Missing, outliers, impossible values, wrong formats, duplicates. An outlier shifts KNN's boundary and corrupts importance.

| Method | Good when / risk |
|---|---|
| Remove rows | huge data, few bad rows; **bias** if not random |
| Impute mean/mode | keeps rows; a guess; train stats |
| Remove column | > 60–70% missing |
| "Is missing" flag | missing may mean something |
| Error correction | age −5, "sixty thousand" |
| Outlier removal | age 250 |
| Duplicate removal | else 5× weight |

**Remove rows?** OK for huge data, few rows. **Risk: bias** if missing isn't random (low incomes skip the question).

**Imputation?** Fill with mean (numbers) or mode (categories), computed on training data.

**Drop a column / "is missing" flag?** Column > 60–70% empty / missing itself may carry meaning.

**Correction, outliers, duplicates?** Fix typos and impossible values (age −5) · delete clear errors (age 250) · delete copies (else 5× weight).

**Why drop ID columns?** No information, distorts distances, memorized → overfitting.

**Feature engineering: why?** Feature engineering = using your knowledge of the topic to **create, change or pick** input columns so the pattern becomes easier to see. Why: a simple model with great features beats a complex model with poor ones; it trains faster and its decisions are easier to explain ("applied ML is basically feature engineering").

**Examples?** Timestamp → hour, weekday, weekend · interaction: study ÷ sleep, size ÷ (distance+1) · binning: age → groups (non-linear patterns, robust to tiny changes).

**Feature selection?** Keep useful, drop irrelevant: better distances, less curse of dimensionality.

## OVERFITTING, LEAKAGE, IMPORTANCE

**Overfitting / underfitting? Overfitting** = the model memorizes the training data (including its noise) instead of the general pattern: great on training, bad on new data (99% train, 60% test), like learning the practice answers by heart. **Underfitting** = too simple to catch the pattern: bad on both.

**Why not train on all data?** Nothing left for an honest grade; split reveals overfitting, builds trust.

**Data leakage?** Information from the test set (or from the future) sneaks into training, so the test score looks great but the model fails in real life, like a student who found the exam answers. Causes: preprocessing before splitting, and leaky features.

**Leak by preprocessing?** Averages/scaling on all data before splitting. Fix: **split first**, compute on train, apply to both.

**Leaky feature?** Not known at prediction time (days since first missed payment). Ask: would I know this then?

**Feature importance: why?** How much the model relies on each feature: trust, feature selection, insight.

**How to read it?** Linear/perceptron: size of weights (on scaled data). Tree: features high up that purify most. **Permutation:** shuffle one test column, accuracy drop = importance (any model).

**random_state / joblib?** Reproducible splits / save model + scaler so new data gets the same transform.

**fit, transform, predict? fit** = learn from data (a model learns weights; a scaler learns mean/std) · **transform** = apply what was learned (scale the data) · **predict** = give answers for new rows. Fit only on training data.

**Safe pipeline?** Split → clean → impute (train) → one-hot → scale (fit train) → train + tune (validation) → test once.

**Deployment risks?** Real data drifts, new categories, unscaled inputs, leaky features missing → monitor and retrain.

## PREDICTED NEW QUESTIONS

**AI ethics? Bias**: learns old discrimination from historical data · **privacy**: needs personal data · **transparency**: black box, people can't see why · **accountability**: who is responsible? "A classifier that flags people is a decision about people." Fix: check data per group, explain, human in the loop.

**Limits of ML?** Needs lots of good, representative data; copies noise and bias; can overfit; hard to explain; fails when the world changes.

**Baseline?** Score of the dumbest model (always the majority class). A model must beat it: 95% is nothing if "always benign" also gets 95%.

**Lower accuracy but better?** On imbalanced data: a model that catches the rare important cases (higher recall) beats one that is "accurate" but finds none.

**F1 (not on slides)?** One number combining precision and recall; high only if both are high.

**Why use the test set only once?** Repeated checking lets it shape the model → optimistic score. Tune on validation; test once at the end.

**Why shuffle before splitting?** A sorted file (all Fails first, by date) would give unequal train and test sets.

**Mean, median or mode?** Mean for normal numbers · median if outliers (a millionaire pulls the mean, not the median) · mode for categories.

**Why is KNN sensitive to outliers/noise?** It decides from a few nearest points; one wrong or extreme point takes a vote (the whole vote with k = 1).

**Why can't a perceptron do XOR?** Positives in opposite corners: no single straight line separates them (not linearly separable) → never converges; needs layers (MLP).

**Why several epochs?** Fixing one point moves the line and can break others, so it loops until a whole epoch has no mistakes.

**Black box / interpretability?** A model whose decisions you can't explain; explaining matters for trust, finding bias and telling affected people why.

**DELETE vs TRUNCATE vs DROP?** Remove chosen rows (table stays) · remove all rows fast (table stays) · remove the whole table.

**ON DELETE CASCADE?** Deleting a parent also deletes its children (SET NULL: sets their FK to NULL). Keeps referential integrity; otherwise the delete is refused.

**Why not email or name as PK?** Names repeat; emails change (every FK must change) and are long. A short surrogate id never changes.

**Weak entity?** Lives only inside its parent → PK(parent_id, own nr): room 101 in many buildings.